

UNIVERSIDAD DE LAS AMÉRICAS PUEBLA

Departamento de Computación, Electrónica y Mecatrónica

Reporte de práctica

Práctica 4: Máquinas de Estado Finito (Control Unit)

Materia	P26-LIS2022-1 Arquitecturas Computacionales
Profesor	Dr. Omar Guillén Fernández
Sección	2
Periodo	Primavera 2026

Integrante

Robbie Nicolás Curioso de Salazar

186028

robbie.curiosodr@udlap.mx



1. Introducción

Las máquinas de estado finito (*Finite State Machines*, FSM) son uno de los modelos más utilizados en el diseño de sistemas digitales secuenciales: permiten describir el comportamiento de un circuito en función de un conjunto reducido de estados, una señal de entrada y una salida que se calcula a partir de ellos. En arquitectura computacional, las FSM aparecen como base de las unidades de control, los detectores de secuencia, los protocolos serie y, en general, en cualquier circuito en el que la respuesta dependa de la historia previa de las entradas.

En esta práctica se diseñaron e implementaron en VHDL tres FSM con propósitos distintos. Las dos primeras son detectores de la secuencia binaria 101 sobre una entrada serial, una con arquitectura Mealy y otra con arquitectura Moore. La tercera es una máquina de seis estados que controla un sistema de semáforo de doble vía con cruce peatonal y que opera directamente con el reloj de 50.000 MHz de la PYNQ Z1, integrando la temporización dentro de su propio contador para producir tiempos de permanencia en escala de segundos sin necesidad de un divisor externo.

El objetivo es comparar el comportamiento temporal de los modelos Mealy y Moore para una misma especificación funcional y, en paralelo, llevar la FSM del semáforo desde la simulación hasta su síntesis y operación física sobre la PYNQ Z1. La verificación funcional se realizó mediante simulación en *Active-HDL* sobre tres bancos de pruebas independientes y, en el caso del semáforo, se complementó con una validación en hardware utilizando Vivado y un prototipo de LED conectado a los conectores PMOD de la tarjeta.

2. Objetivos

- Diseñar una FSM tipo Mealy que detecte la secuencia binaria 101 sobre una entrada serial.
- Diseñar una FSM tipo Moore que detecte la misma secuencia y comparar su temporización con la versión Mealy.
- Implementar en VHDL un sistema de semáforo de doble vía con cruce peatonal mediante una FSM de seis estados.
- Integrar la temporización dentro del contador interno de la FSM para que cada estado permanezca activo el número de segundos especificado, partiendo del reloj de 50.000 MHz de la PYNQ Z1.

- Sintetizar el diseño en Vivado, asignar los pines mediante un archivo `xdc` y validar el funcionamiento del semáforo sobre la PYNQ Z1 con un prototipo de LED conectado a los PMOD.
- Validar mediante simulación en Active-HDL los tres diseños y contrastar los resultados con el comportamiento teórico esperado.

3. Materiales y herramientas

- **Software de simulación:** Active-HDL Student Edition para compilación y simulación funcional.
- **Software de síntesis:** Vivado Design Suite para síntesis, implementación y generación del *bitstream* cargado en la PYNQ Z1.
- **Lenguaje:** VHDL, con uso de las bibliotecas `ieee.std_logic_1164` y `ieee.numeric_std`.
- **Plataforma objetivo:** tarjeta de desarrollo PYNQ Z1 (Zynq 7000), reloj de placa de 50.000 MHz.
- **Hardware del prototipo:** protoboard, LED de cada color (rojo, ámbar, verde y azul para los bits de estado), resistencias limitadoras de corriente y cables *jumper* para conectar los PMOD de la PYNQ Z1.
- **Documentación:** guía oficial de la práctica 4, apuntes del curso sobre FSM Mealy/-Moore, hoja de datos de la PYNQ Z1 (mapa de pines) y la documentación de Active-HDL y Vivado.

4. Marco teórico

4.1. Máquinas de estado finito

Una FSM es una abstracción de un circuito secuencial síncrono compuesto por una memoria de estado (un registro), una función de transición y una función de salida. En cada flanco activo del reloj, el estado actual se actualiza al siguiente estado calculado a partir de la entrada y del estado actual. La salida puede depender únicamente del estado (modelo Moore) o del estado y de la entrada (modelo Mealy).

4.2. Modelo Mealy

En una FSM Mealy la salida es función combinacional del estado actual y de la entrada presente, lo que se expresa como $y = g(s, x)$. Esto permite que la salida cambie en el mismo ciclo en el que se completa la secuencia de interés y, en general, conduce a diseños con menos estados. Sin embargo, la salida hereda la sensibilidad combinacional a la entrada, por lo que puede presentar pequeños transitorios (*glitches*) entre flancos del reloj.

4.3. Modelo Moore

En una FSM Moore la salida depende exclusivamente del estado actual, $y = g(s)$. Esto introduce un retardo de un ciclo entre la llegada del último bit de la secuencia y la activación de la salida, ya que el patrón debe primero conducir a un estado de aceptación. La ventaja es que la salida queda registrada al estado y no presenta transitorios derivados de la entrada, lo que la hace más segura cuando se conecta a otros bloques registrados.

4.4. Detector de la secuencia 101

El detector de la secuencia binaria 101 es un ejemplo clásico para comparar ambos modelos. La especificación es: cada vez que en la entrada serial aparezca el patrón 1,0,1 (en ese orden y en ciclos consecutivos del reloj), debe activarse la salida **flag**. Para permitir detecciones traslapadas, la máquina no se reinicia al estado inicial después de cada acierto. La versión Mealy se construye con tres estados (S0 reposo, S1 vio un 1, S2 vio 10); la versión Moore añade un cuarto estado S3 que representa la detección completa y donde se activa la salida.

4.5. Semáforo de doble vía con cruce peatonal

El sistema modela un cruce con una vía principal (calle A), una vía secundaria (calle B) y un cruce peatonal. Las luces vehiculares se codifican con tres bits, uno por cada color (rojo=001, ámbar=010, verde=100), mientras que la luz peatonal se codifica con dos bits (rojo=01, verde=10, parpadeo=11). El comportamiento se describe con seis estados que se recorren cíclicamente: dos fases vehiculares (A y B), cada una seguida de una fase de ámbar para liberar el cruce, y dos fases peatonales (verde y parpadeo de aviso). El sistema admite además una entrada de emergencia que fuerza el retorno al estado inicial y deja todas las luces vehiculares en rojo y la peatonal en rojo.

4.6. Temporización a partir del reloj de placa

La FSM del semáforo trabaja en escala de segundos, mientras que el reloj físico de la PYNQ Z1 es de 50.000 MHz. Una primera opción para conciliar ambas escalas consiste en utilizar un divisor de frecuencia que genere un reloj de 1.000 Hz a partir del reloj de placa (contando 25 000 000 ciclos del reloj de 50.000 MHz y conmutando la salida en cada cuenta completa) y conectar dicho divisor a la FSM mediante un módulo estructural de nivel superior. En esta práctica se siguió un enfoque equivalente pero más compacto: la FSM se alimenta directamente del reloj de 50.000 MHz y la temporización se logra dentro del propio contador interno, configurando `timeout_value` en múltiplos de 50 000 000 ciclos (uno por cada segundo de permanencia en cada estado). Este enfoque elimina la necesidad de un módulo `top` estructural y reduce el diseño a una única entidad sintetizable.

5. Diseño y procedimiento

5.1. Detector Mealy de la secuencia 101

La entidad `mealy_101` declara los puertos `clk`, `rst`, `input_bit` y `flag`. El registro de estado se actualiza en un proceso síncrono a `clk` con `rst` asíncrono que regresa la máquina a S0. La función de transición y la salida se calculan en un proceso combinacional. Las transiciones programadas son: en S0, si `input_bit=1` se pasa a S1, si no se queda en S0; en S1, si `input_bit=0` se pasa a S2, si no se queda en S1; en S2, si `input_bit=1` se activa `flag` y se regresa a S1 (para permitir detecciones traslapadas), si no se regresa a S0. Como en Mealy la salida depende también de la entrada, `flag` se activa exactamente en el ciclo en el que llega el tercer bit de la secuencia.

5.2. Detector Moore de la secuencia 101

La entidad `moore_101` tiene la misma interfaz que la versión Mealy, pero usa cuatro estados: S0, S1, S2 y un estado adicional S3 de detección completa. El registro de estado se actualiza con la misma estructura síncrona y el proceso combinacional define las transiciones siguientes: S0 con `input_bit=1` pasa a S1; S1 con `input_bit=0` pasa a S2; S2 con `input_bit=1` pasa a S3; y S3 con `input_bit=0` regresa a S2 o, con `input_bit=1`, regresa a S1, también para permitir detecciones traslapadas. La salida `flag` se asigna directamente al estado S3 dentro del proceso combinacional. Como consecuencia, la salida se activa un ciclo después del último

bit de la secuencia, comportamiento característico de Moore.

5.3. FSM del semáforo

La entidad `fsm_semaforo` expone como entradas `clk`, `rst` y `emergency`, y como salidas tres vectores de luces (`luz1`, `luz2`, `luz3`) más un vector de seis bits `state` para observar el estado durante la simulación. Los seis estados se codifican *one-hot* con seis bits (`S0=000001...S5=100000`). La asignación de luces por estado se resume en la Tabla 1 y los tiempos de permanencia en la Tabla 2.

Cuadro 1: Asignación de luces por estado del semáforo.

Estado	Calle A (luz1)	Calle B (luz2)	Peatonal (luz3)
S0	Verde	Rojo	Rojo
S1	Ámbar	Rojo	Rojo
S2	Rojo	Verde	Rojo
S3	Rojo	Ámbar	Rojo
S4	Rojo	Rojo	Verde
S5	Rojo	Rojo	Parpadeo

Cuadro 2: Tiempo de permanencia por estado y valor de `timeout_value` (ciclos del reloj de 50.000 MHz).

Estado	Tiempo (s)	<code>timeout_value</code> (ciclos)
S0	30	$30 \times 50\,000\,000$
S1	5	$5 \times 50\,000\,000$
S2	20	$20 \times 50\,000\,000$
S3	5	$5 \times 50\,000\,000$
S4	15	$15 \times 50\,000\,000$
S5	10	$10 \times 50\,000\,000$

Internamente la FSM utiliza un contador de 32 bits y una señal `timeout_value` que se ajusta combinatorialmente al valor asociado al estado actual. En cada flanco de `clk`, si `emergency=0`, el contador incrementa hasta alcanzar `timeout_value`, momento en el que la máquina pasa al siguiente estado y el contador se reinicia. Si `emergency=1`, el contador se

limpia y la máquina regresa a S0; en ese mismo ciclo el proceso de salidas pone todas las luces vehiculares en rojo y la peatonal en rojo, llevando el sistema a una condición segura. Como la FSM se alimenta directamente del reloj de 50.000 MHz de la PYNQ Z1 y `timeout_value` se programa en ciclos de ese reloj, los tiempos de la Tabla 2 se traducen directamente en segundos sobre la tarjeta sin necesidad de un divisor de frecuencia separado.

5.4. Flujo en Active-HDL

Para cada simulación se siguió el procedimiento habitual:

1. Crear un espacio de trabajo y agregar al diseño los archivos VHDL correspondientes (`mealy_101.vhd`, `moore_101.vhd` o `fsm_semaforo.vhd`).
2. Compilar la entidad correspondiente.
3. Inicializar la simulación con la entidad como nivel superior y aplicar los estímulos desde el panel de *Stimulators* de Active-HDL.
4. Agregar al *waveform* las señales relevantes y configurar formato binario para las entradas seriales y para los vectores de luces, y formato hexadecimal para los registros de estado.
5. Ejecutar *Run For* con la duración necesaria para observar todas las transiciones.

6. Resultados y discusión

6.1. Detector Mealy de la secuencia 101

La Figura 1 muestra la simulación del detector Mealy. Tras un pulso inicial de `rst`, la máquina parte de S0 y avanza a S1 cuando `input_bit` toma el valor 1. Conforme la cadena de prueba aporta los bits 0 y 1, la FSM transita por S2 y vuelve a S1, activando `flag` en el mismo ciclo en el que llega el tercer bit. En la cadena 010110101101 esta condición se cumple en varias ocasiones, generando pulsos en `flag` alineados con el último bit de cada ocurrencia de 101.

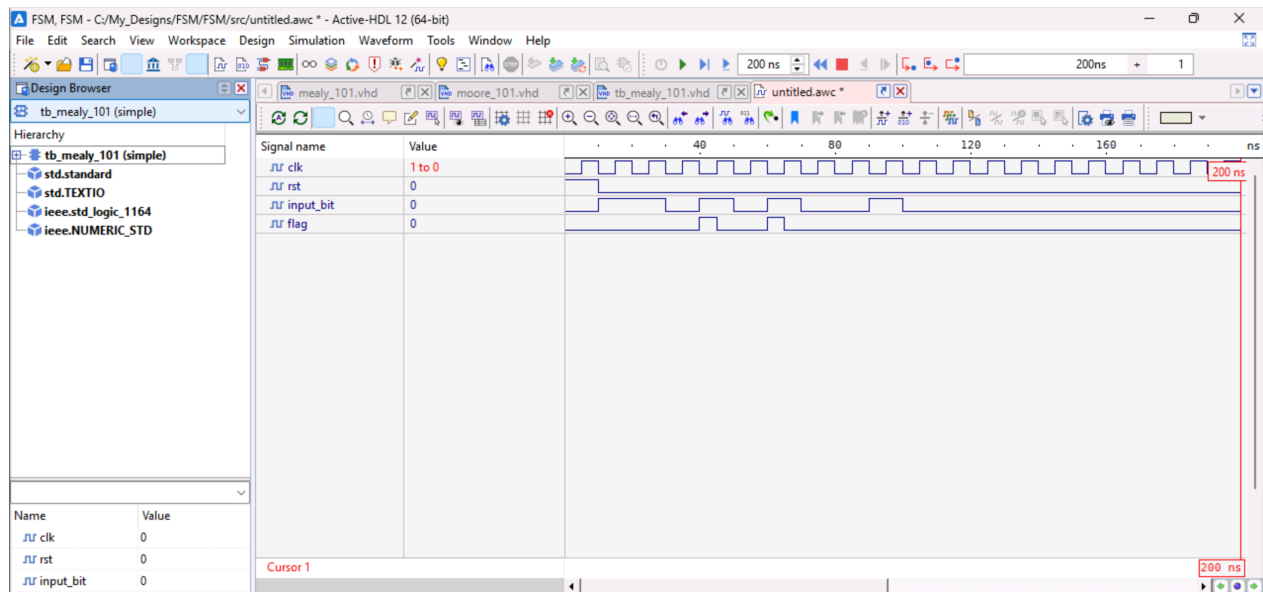


Figura 1: Simulación del detector Mealy: la salida **flag** se activa en el mismo ciclo en el que llega el último 1 de la secuencia.

Este comportamiento confirma la propiedad fundamental del modelo Mealy: la salida es función combinacional de estado y entrada. Por ello **flag** sigue de cerca a **input_bit** cuando la máquina está en S2, y vuelve a cero en cuanto la entrada cambia. La máquina no requiere un estado dedicado a la aceptación, lo cual reduce el conteo a tres estados.

6.2. Detector Moore de la secuencia 101

La Figura 2 muestra la simulación del detector Moore. La máquina recorre los mismos estados intermedios que la versión Mealy, pero la salida **flag** solo se activa cuando se entra al estado S3, es decir, en el ciclo posterior al último 1 de la secuencia. Esto se observa con claridad al alinear los pulsos de **flag** con **input_bit**: cada activación aparece desplazada un período de reloj respecto al instante en el que se completó el patrón.

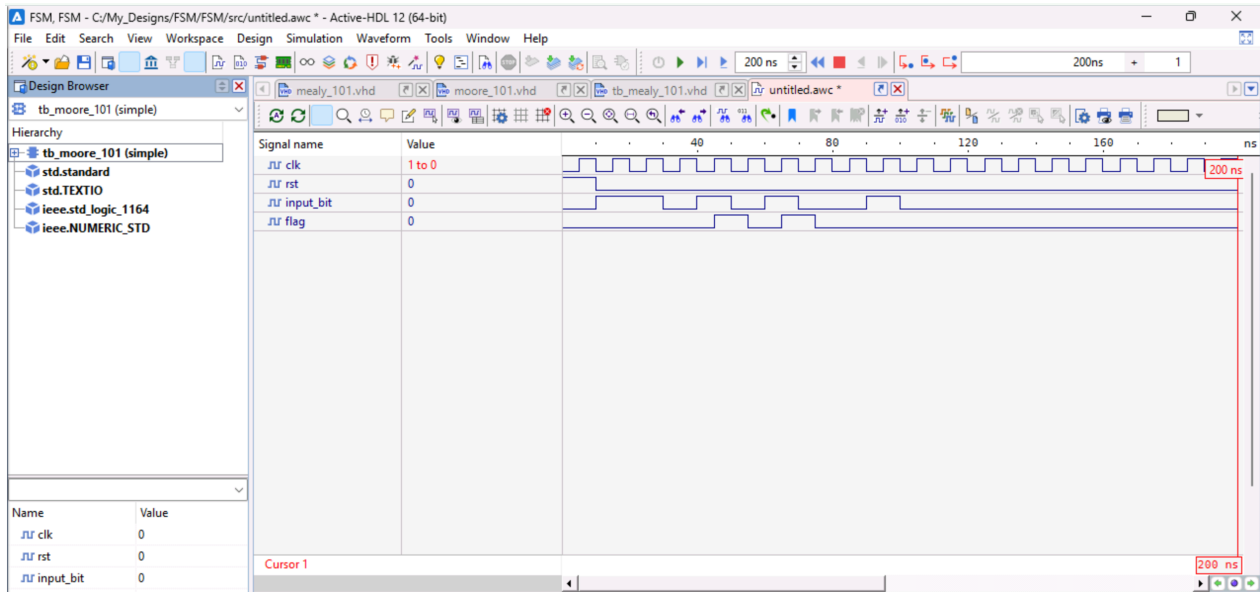


Figura 2: Simulación del detector Moore: **flag** se activa al entrar al estado S3, un ciclo después del último bit de la secuencia.

La introducción del estado S3 hace que la salida quede registrada al estado y no presente transitorios derivados de la entrada, a costa de un retardo de un ciclo. Esta propiedad es precisamente la que se busca cuando la salida de la FSM se conecta a otro bloque registrado, ya que evita carreras críticas y facilita el cumplimiento de los tiempos de *setup*.

6.3. FSM del semáforo

A diferencia de los detectores Mealy y Moore, la verificación de la FSM del semáforo no se realizó por simulación sino directamente sobre el arreglo experimental montado en la PYNQ Z1, ya que los tiempos de permanencia de cada estado (decenas de segundos cuando el contador se programa en múltiplos de 50 000 000 ciclos) hacen poco práctico observarlos en un *waveform* y son fáciles de validar a simple vista en el prototipo de LED. Conceptualmente el comportamiento esperado es el siguiente: la FSM avanza ordenadamente por los estados S0, S1, S2, S3, S4 y S5, codificados como 01, 02, 04, 08, 10 y 20 en hexadecimal por la codificación *one-hot*, y regresa cíclicamente a S0. La señal **counter** incrementa en cada flanco activo y dispara la transición cuando alcanza **timeout_value**, valor que se ajusta combinatorialmente al estado en curso, mientras que las salidas **luz1**, **luz2** y **luz3** cambian de acuerdo con la Tabla 1.

Al activar la entrada **emergency**, la FSM regresa al estado S0 y la asignación de salidas deja **luz1=001**, **luz2=001** y **luz3=01**, condición segura en la que ambas vías quedan en rojo y el

peatón en rojo. Esto valida el camino de control de emergencia y se observa de inmediato sobre el prototipo al pulsar el botón asignado a esta señal, como se discute en la siguiente sección.

7. Implementación en hardware sobre PYNQ Z1

Una vez verificada funcionalmente la FSM del semáforo en Active-HDL, el diseño se sintetizó con Vivado y se programó en la tarjeta de desarrollo PYNQ Z1. Como se discutió en el marco teórico, en la implementación final no se utilizó un módulo estructural de nivel superior con un divisor de frecuencia separado: el contador interno de la FSM se ajustó para contar $N \times 50\,000\,000$ ciclos del reloj de 50.000 MHz de la placa, de manera que cada cuenta completa corresponde directamente a N segundos de permanencia en el estado. De esta forma, el diseño físico mantiene la jerarquía mínima (una única entidad `fsm_semaforo`) y se programan los tiempos del semáforo modificando únicamente las constantes del proceso combinacional que asigna `timeout_value`.

7.1. Asignación de pines (constraints)

El archivo `semaforo.xdc` describe la asignación de los puertos de la entidad a pines físicos de la PYNQ Z1. El reloj de placa de 50.000 MHz se conecta al pin H16, los botones de la tarjeta se utilizan para reset y emergencia, y las luces vehiculares y peatonal se enrutan a los conectores PMOD para alimentar el prototipo de LED. Los seis bits del vector `state` también se exponen como salidas para observar la codificación *one-hot* de la FSM en tiempo real durante la operación. La Tabla 3 resume el mapeo principal.

Cuadro 3: Mapeo principal del archivo `semaforo.xdc` sobre la PYNQ Z1.

Señal	Pin (paquete)	Función física
<code>clk</code>	H16	Reloj de placa de 50.000 MHz
<code>rst</code>	D19	Botón BTN0 (reset)
<code>emergency</code>	L19	Botón BTN3 (emergencia)
<code>luz1[2:0]</code>	Y18, Y19, Y16	Calle A: verde, ámbar, rojo
<code>luz2[2:0]</code>	U18, U19, W18	Calle B: verde, ámbar, rojo
<code>luz3[1:0]</code>	Y17, W19	Peatón: verde, rojo
<code>state[5:0]</code>	W14, Y14, T11, T10, V16, W16	Estado one-hot (depuración)

Todos los pines se declararon con el estándar `LVCMS33`, compatible con la tensión de los conectores PMOD de la PYNQ Z1 y con los LED utilizados en el prototipo. Adicionalmente, en el constraint del reloj se definió un período de 20,000 ns con la directiva `create_clock`, que corresponde a la frecuencia nominal de 50.000 MHz.

7.2. Prototipo de LED y operación física

El prototipo se montó sobre una protoboard, con un LED por cada salida del semáforo y una resistencia limitadora de corriente en serie con cada uno. Las tres salidas de la calle A (`luz1`) se asociaron a un LED rojo, uno ámbar y uno verde; las tres salidas de la calle B (`luz2`) siguieron el mismo esquema, y la salida peatonal (`luz3`) se montó con un LED rojo y uno verde. Los seis bits del vector `state` se conectaron a una segunda hilera de LED para visualizar la codificación *one-hot* del estado activo en cada momento.

Las Figuras 3 y 4 muestran al prototipo en operación durante dos instantes distintos del ciclo del semáforo. En la Figura 3 se observa una combinación de luces vehiculares y peatonales correspondiente a una de las fases intermedias del ciclo, junto con varios bits del vector de estado encendidos en los LED de depuración. La Figura 4 corresponde a otro instante del mismo ciclo, con una combinación distinta de luces y un único bit del vector `state` activo, lo cual confirma la codificación *one-hot* usada por la FSM.

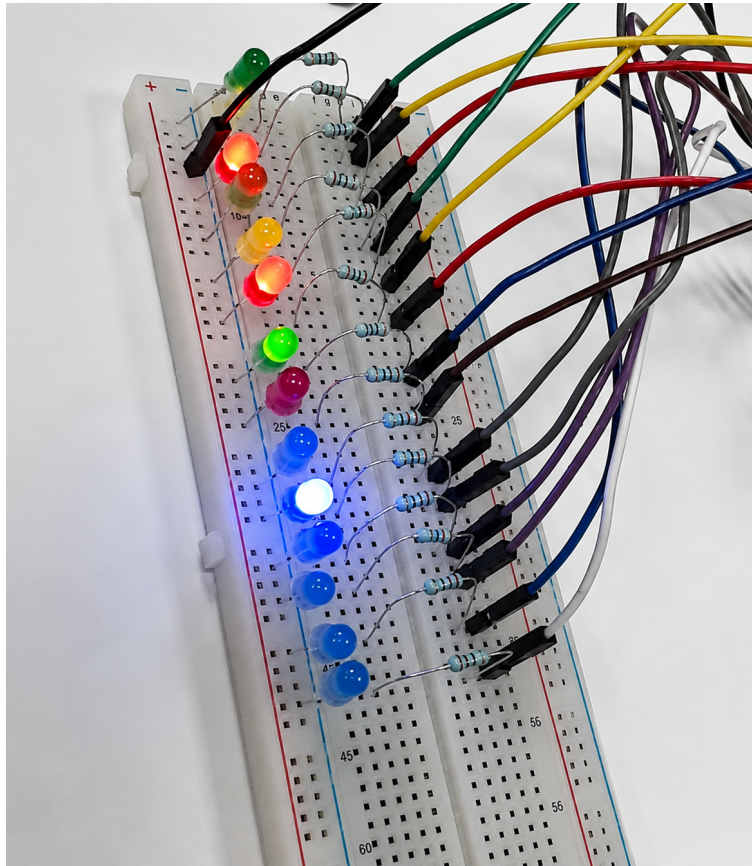


Figura 3: Prototipo del semáforo en operación sobre la PYNQ Z1: luces vehiculares de calle A y calle B, luz peatonal y bits del vector `state` usados como depuración.

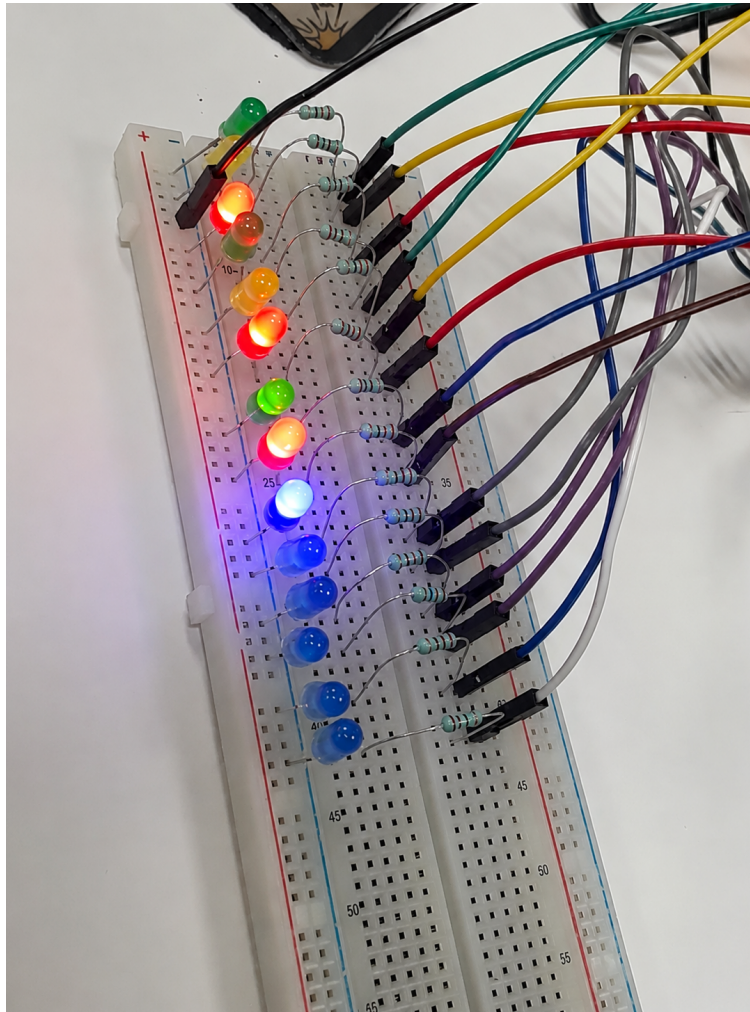


Figura 4: Otro instante del ciclo del semáforo sobre la PYNQ Z1: la combinación de luces y los LED de estado confirman una fase distinta de la FSM.

Durante la prueba se verificó cualitativamente que cada estado se mantenía durante el tiempo programado en `timeout_value` (30, 5, 20, 5, 15 y 10 segundos para S0–S5, respectivamente), lo cual fue posible gracias al ajuste del contador interno al reloj real de 50.000 MHz de la placa. Al pulsar el botón asignado a **emergency** (BTN3), la FSM regresaba inmediatamente al estado S0 con todas las luces vehiculares en rojo y la peatonal en rojo, replicando en hardware el camino de control de emergencia validado previamente en simulación. El botón BTN0 (reset) reiniciaba la máquina al estado inicial sin importar la fase en curso.

8. Análisis de resultados

La comparación entre las dos arquitecturas del detector 101 confirma las diferencias teóricas estudiadas en el curso. La FSM Mealy resuelve la detección con tres estados y reacciona en el mismo ciclo en el que llega el último bit; la FSM Moore necesita cuatro estados pero produce una salida estable y registrada con un retardo de un ciclo. En sistemas pequeños como éste, ambas opciones son válidas, pero la elección depende del entorno: cuando la salida alimenta lógica combinacional sensible a transitorios, conviene Moore; cuando se requiere mínima latencia, conviene Mealy. La forma en que se permitieron las detecciones traslapadas (regresar a S1 desde S2 o S3 al recibir un 1) es la misma en ambos casos y se comprueba en la simulación al observar dos activaciones consecutivas de `flag` sobre la subcadena 10101.

En la FSM del semáforo, la relación entre `current_state`, `next_state` y `timeout_value` muestra que la transición entre estados no ocurre de forma inmediata sino únicamente cuando el contador alcanza el tiempo asignado al estado actual. Este comportamiento confirma que la temporización fue integrada correctamente dentro de la máquina y que cada fase permanece activa durante el intervalo programado. En la implementación física sobre la PYNQ Z1, `timeout_value` se programa en múltiplos de 50 000 000 ciclos de manera que cada cuenta corresponde a un segundo real, lo cual se verificó cualitativamente durante la operación del prototipo de LED descrito en la sección anterior. En simulación, en cambio, los tiempos se interpretan directamente como ciclos del reloj de prueba para acortar el tiempo de cómputo.

Las salidas observadas para las luces vehiculares y peatonales coinciden con la asignación planeada en cada estado. La codificación *one-hot* del estado facilitó la lectura del *waveform* y, en una eventual síntesis, permitiría implementar el registro de estado con seis biestables individuales, lo cual suele ser eficiente en FPGA.

9. Buenas prácticas

Durante el desarrollo se mantuvieron las convenciones que ya se habían adoptado en prácticas anteriores: separación clara entre lógica secuencial y combinacional, asignaciones por defecto al inicio de los procesos combinacionales para evitar latches no deseados, uso exclusivo de `numeric_std` para aritmética sobre vectores y descripción de los módulos con interfaces mínimas. La codificación *one-hot* del estado en la FSM del semáforo se eligió por simplicidad de lectura en el *waveform* y porque suele inferir registros eficientes en FPGA.

10. Conclusiones

Se diseñaron y verificaron en VHDL tres FSM con propósitos distintos: dos detectores de la secuencia 101 (uno Mealy y otro Moore) y una unidad de control para un semáforo de doble vía con cruce peatonal. La simulación en Active-HDL confirmó las diferencias de temporización entre los modelos Mealy y Moore: el primero activa la salida en el mismo ciclo en el que se completa la secuencia, mientras que el segundo lo hace un ciclo después al depender únicamente del estado. Ambas implementaciones permiten detecciones traslapadas y son consistentes con el comportamiento teórico esperado.

La FSM del semáforo recorrió correctamente los seis estados, respetó los tiempos asignados a cada fase mediante el contador interno y la señal `timeout_value`, y dejó preparado el camino de control de emergencia. Posteriormente, el diseño se sintetizó con Vivado y se programó en la PYNQ Z1 utilizando un archivo de *constraints* que asigna el reloj de placa, los botones BTN0 y BTN3 y las salidas de luces a los conectores PMOD. En el prototipo físico, los tiempos definidos en `timeout_value` (programados en múltiplos de 50 000 000 ciclos del reloj de 50.000 MHz) se tradujeron directamente en segundos reales, y el botón de emergencia llevó al sistema a un estado seguro de manera inmediata, completando así el flujo desde la simulación hasta la operación sobre hardware.

Esta práctica reforzó el manejo de procesos secuenciales y combinacionales en VHDL, así como la importancia de las FSM como base de las unidades de control en arquitecturas computacionales.

11. Referencias

Referencias

- [1] O. Guillén Fernández. *Práctica 4: Máquinas de Estado Finito (Control Unit)*. Universidad de las Américas Puebla.
- [2] P. J. Ashenden. *The Designer's Guide to VHDL*. Morgan Kaufmann, 2008.
- [3] D. M. Harris and S. L. Harris. *Digital Design and Computer Architecture*. Morgan Kaufmann, 2nd ed., 2012.
- [4] IEEE. *IEEE Standard VHDL Language Reference Manual*. IEEE Std 1076.

- [5] Aldec. *Active-HDL Documentation*. Disponible en: https://www.aldec.com/en/products/fpga_simulation/active_hdl
- [6] Xilinx. *UG585: Zynq-7000 SoC Technical Reference Manual*. Disponible en: https://www.xilinx.com/support/documentation/user_guides/ug585-Zynq-7000-TRM.pdf

Repositorio del proyecto

El código fuente VHDL de las tres FSM, junto con las capturas de las formas de onda obtenidas en Active-HDL, se encuentra disponible en el siguiente repositorio público de GitHub:

<https://github.com/Robbienicur/Arquitecturas-Computacionales/tree/main/Practica-4>

La carpeta Practica-4 del repositorio contiene los archivos `mealy_101.vhd`, `moore_101.vhd` y `fsm_semaforo.vhd`, los respectivos *testbenches*, el archivo de *constraints* `semaforo.xdc` para la PYNQ Z1, las imágenes de simulación y del prototipo físico (`Proto1.png` y `Proto2.png`), y este mismo reporte en formato PDF.